Guide technique

Projet Signaux · suivi de consommation par lecture de compteur · À quoi sert chaque fichier, méthodes à écrire, fonctions et docs à aller voir — sans code
Version du 21 octobre 2026 · même contenu que le wiki du dépôt

À quoi sert chaque fichier

> ⚠️ Avant de commencer : faire un **Pull**. Le commit « flask » sur GitHub contient `app.py` et une première version de `index.html`, qui ne sont peut-être pas encore sur votre PC.

Ce qui existe déjà

Fichier / dossier	À quoi il sert	État
`app.py`	Le site Flask, écrit sous forme d'une classe `Application`. Contient les routes `/` (la page), `/api/data` (les données en JSON) et `/upload` (pour le Pico, plus tard).	Squelette : les méthodes renvoient des données vides
`main.py`	Ancien point d'entrée.	⚠️ Plante : `response` n'existe pas dans Flask (c'est `Response` ou `make_response`), et `sqlite3` n'est plus utile avec Supabase. **Décider** : soit on le supprime, soit il sert seulement à lancer `Application`. Un seul point d'entrée.
`templates/index.html`	La page web. Utilise Chart.js. Contient une variable `MODE_TEST` : `true` = données bidon, `false` = vraies données venant de `/api/data`.	Fonctionne en mode test
`static/css/style.css`	Le style de la page.	Vide
`static/js/script.js`	Le JavaScript de la page (on peut y déplacer le script de `index.html`).	Vide
`classes/`	Nos classes Python (POO). `__init__.py` permet d'écrire `from classes import ...`.	Vide
`MicroPython/main.py`	Le code du Pico (plan B).	Vide, normal
`wiki/`	Ces pages, à publier dans l'onglet Wiki de GitHub.	—
`ressources/`	Les PDF : consignes, plan, guide technique.	—
`.idea/`	Réglages de PyCharm. Ne pas y toucher.	—

Ce qu'il reste à créer

Fichier	À quoi il sert	Qui
`.env`	Le mot de passe / l'adresse de connexion Supabase. **Jamais dans Git.**	D
`.gitignore`	Dit à Git d'ignorer `.env`, `__pycache__/`, `venv/`.	D
`requirements.txt`	La liste des bibliothèques à installer (`flask`, `opencv-python`, `python-dotenv`, un client Postgres ou Supabase).	D
`static/images/`	Les photos du compteur.	A
Un fichier CSV	Une ligne par photo : nom du fichier, date, vrai index, condition.	A
Les classes ci-dessous	Le cœur du programme.	B, C, D

Les classes proposées

Ce sont des **suggestions de noms et de rôles**. À vous d'écrire le contenu. Une classe = une responsabilité = un fichier dans `classes/`.

`Releve` — une photo et ce qu'on sait dessus (D)

Représente une ligne de la table.

- **Attributs** : `image`, `date_releve`, `vrai_index`, `index_lu`, `condition`.
- **Indice** : regarder `@dataclass` (module `dataclasses`) : ça évite d'écrire `__init__` à la main.
- **Méthode utile** : `est_juste()` → vrai si `index_lu` égale `vrai_index`, rien si l'un des deux manque.

`BaseDeDonnees` — la seule classe qui parle à Supabase (D)

- `__init__()` : lit l'adresse de connexion dans `.env`.
- `tous_les_releves()` → liste de `Releve` triés par date.
- `releves_sans_lecture()` → ceux dont `index_lu` est vide.
- `ajouter_releve(releve)` → ajoute une photo (ou la met à jour si elle existe déjà).
- `enregistrer_index_lu(releve)` → sauvegarde ce que le programme a lu.

`LecteurCompteur` — photo → nombre (B et C)

- `__init__(dossier_images, zone)` : `zone` = les coordonnées de l'écran du compteur sur la photo.
- `pretraiter(nom_image)` → image noir et blanc, chiffres en blanc (B).
- `decouper_chiffres(image)` → liste de petites images, une par chiffre (B).
- `reconnaitre_chiffre(image_chiffre)` → un chiffre de 0 à 9 (C).
- `lire(nom_image)` → l'index complet, ou rien si la lecture échoue (C).

`AnalyseConsommation` — les calculs (D)

- `__init__(releves, source)` : `source` dit si on utilise l'index lu ou le vrai index.
- `consommations()` → pour chaque intervalle : la date et `index − index précédent`. Rejette les index qui baissent.
- `statistiques()` → moyenne, minimum, maximum (pic) et sa date, total.
- `taux_de_reussite()` → % de photos où `index_lu` = `vrai_index`.

`Application` — le site (D, déjà commencée dans `app.py`)

- `getPage()` → renvoie la page.
- `getData()` → utilise `BaseDeDonnees` puis `AnalyseConsommation`, renvoie le JSON pour le graphique.
- `upload()` → plan B.

Comment les morceaux s'enchaînent

```
photos + CSV ──> BaseDeDonnees.ajouter_releve()        (A)
                          │
LecteurCompteur.lire() ───┼──> BaseDeDonnees.enregistrer_index_lu()   (B, C)
                          │
Application.getData() ────┴──> AnalyseConsommation ──> JSON ──> Chart.js   (D)
```

Pas besoin d'aller plus loin en POO : pas d'héritage, pas d'interfaces. Pour un projet de 2 semaines, 4 ou 5 classes simples suffisent.

Base de données (Supabase)

Une seule table, appelée `releves`. Tout le reste (consommation, moyenne, minimum, pic, taux de réussite) se **calcule** en Python : on ne le stocke pas.

Les colonnes

Colonne	Type Postgres à utiliser	Qui la remplit	À quoi elle sert
`id`	entier auto-incrémenté, clé primaire	la base	identifiant
`image`	texte, unique, obligatoire	A (import)	nom du fichier dans `static/images/`
`date_releve`	date + heure avec fuseau (`timestamptz`)	A (import)	moment de la photo
`vrai_index`	nombre décimal (`numeric`)	A, **à la main**	la bonne réponse, pour vérifier le programme
`index_lu`	nombre décimal (`numeric`), peut être vide	le programme	ce que notre lecture a trouvé
`condition`	texte	A	`jour`, `lampe`, `faible`, `incline`, `flou` ou `reflet`

Pourquoi pas moins de colonnes ?

- Sans `vrai_index`, impossible de prouver que le programme lit juste. Or la validation est demandée.
- Sans `condition`, impossible de faire le tableau « ça marche dans quelles conditions ».

Pourquoi les photos ne sont pas dans la base ?

Une image dans une table, c'est lourd et lent. La photo reste dans `static/images/` (Flask l'affiche directement) et la table garde seulement son **nom**.

Créer la table

Deux façons, au choix :

- **À la souris** : Supabase › *Table Editor* › *New table*, puis ajouter les colonnes du tableau ci-dessus.
- **En SQL** : Supabase › *SQL Editor*. Mots-clés à chercher : `create table`, `generated always as identity primary key`, `unique`, `not null`, `numeric(10,2)`, `timestamptz`.

Ensuite, garder le SQL de création dans un fichier du projet (par exemple `database/schema.sql`) : c'est utile pour le README et le jury.

Se connecter depuis Python

- Dans Supabase, bouton **Connect** : copier l'adresse **Session pooler** (elle marche sur tous les réseaux).
- La mettre dans `.env`, la lire avec `python-dotenv` (`load_dotenv()`, puis `os.environ`).
- Deux choix de bibliothèque :

- **psycopg** (on écrit du SQL soi-même) : `psycopg.connect()`, `execute()`, `fetchall()`. Regarder `row_factory=dict_row` pour avoir des dictionnaires.
- **supabase-py** (pas de SQL) : `create_client()`, `.table("releves").select()`, `.insert()`, `.update()`, `.upsert()`.

Requêtes dont on aura besoin

Besoin	Mots-clés SQL
Tous les relevés dans l'ordre	`select … order by date_releve`
Ceux pas encore lus	`where index_lu is null`
Ajouter sans doublon	`insert … on conflict (image) do update`
Sauver l'index lu	`update … set index_lu = … where image = …`
Taux de réussite par condition	`group by condition`, `count(*) filter (where …)`

Sécurité

- Le mot de passe de la base ne va **jamais** sur GitHub ni dans le ZIP Moodle. Rendre un `.env.example` vide à la place.
- Supabase conseille d'activer **RLS** (Row Level Security) sur la table. Comme notre code se connecte avec le mot de passe de la base, il n'est pas bloqué.

Prendre et annoter les photos (rôle A)

Trouver un compteur

- Un compteur électrique avec un **écran à chiffres 7 segments** (chiffres en petits bâtons).
- À éviter : les compteurs à rouleaux mécaniques et les compteurs intelligents dont l'écran s'éteint ou défile.
- Si personne n'en a : une **prise wattmètre** (prise avec un petit écran) marche aussi, et on peut faire varier la consommation en branchant une bouilloire ou un radiateur. C'est pratique pour avoir un vrai **pic** dans la démo.

Prendre les photos

- Téléphone **posé ou sur un trépied**, toujours **au même endroit**, de face, à 20–40 cm.
- Une photo toutes les 5 minutes : une appli de **time-lapse** ou d'**intervalle** le fait toute seule.
- Le compteur avance lentement : étaler les séances sur plusieurs heures ou jours.
- Varier **exprès** les conditions, pour tester la robustesse : jour, lampe, faible lumière, léger angle, léger flou, reflet.

Objectif : au moins 150 photos.

Nommer les fichiers

Mettre la date et l'heure dans le nom : `2026-10-22_15-30.jpg`. Pas de doublon possible et l'ordre est évident. Les ranger dans `static/images/`.

Annoter : le fichier CSV

Une ligne par photo. Le **vrai index** est lu **à l'œil** sur la photo.

image	date_releve	vrai_index	condition
2026-10-22_15-30.jpg	2026-10-22 15:30:00+02	168.80	jour
2026-10-22_15-35.jpg	2026-10-22 15:35:00+02	168.90	lampe

- `+02` = heure d'été belge, `+01` = heure d'hiver (à partir du 25 octobre).
- `condition` : un seul mot parmi `jour`, `lampe`, `faible`, `incline`, `flou`, `reflet`.
- Le point comme séparateur décimal (168.80, pas 168,80).

Mettre de côté les photos de test

Avant que B et C commencent à régler leur programme : choisir 1 photo sur 3 comme **photo de test**. On ne s'en sert pas pendant le réglage, seulement à la fin pour mesurer le résultat. Sinon, on triche sans le vouloir.

Importer dans la base

Écrire un petit script qui lit le CSV et ajoute chaque ligne dans la table.

- Lire le CSV : module `csv`, classe `DictReader`.

- Transformer le texte en date : `datetime.fromisoformat()`.
- Vérifier que la photo existe : `pathlib.Path(...).exists()`.
- Ajouter dans la base : la méthode `ajouter_releve()` de la classe `BaseDeDonnees`.

Traitement d'image (rôles B et C)

But : transformer une photo du compteur en un nombre (l'index). Tout se passe dans la classe `LecteurCompteur`. On utilise **OpenCV** (`pip install opencv-python`, puis `import cv2`).

Étape 1 — Recadrer sur les chiffres (B)

Les photos sont prises toujours au même endroit, donc l'écran est toujours au même endroit sur l'image. On règle la zone **une seule fois**.

- Ouvrir une photo dans Paint ou GIMP, noter les coordonnées du coin haut-gauche et bas-droite de l'écran.
- En Python, une image OpenCV est un tableau : recadrer = `image[y1:y2, x1:x2]` (attention : **y d'abord**).
- Fonction pour charger : `cv2.imread()`. Pour voir le résultat : `cv2.imshow()` + `cv2.waitKey()`, ou `cv2.imwrite()` pour sauver.

Étape 2 — Noir et blanc (B)

Ce qu'on fait	Fonction OpenCV	Pourquoi
Passer en gris	`cv2.cvtColor()` avec `cv2.COLOR_BGR2GRAY`	La couleur ne sert à rien ici
Adoucir le bruit	`cv2.GaussianBlur()`	Enlève les petits points parasites
Seuillage	`cv2.threshold()` avec `cv2.THRESH_OTSU`	OpenCV choisit seul le bon seuil
Inverser si besoin	option `cv2.THRESH_BINARY_INV`	On veut des chiffres **blancs** sur fond **noir**

Si la lumière varie beaucoup sur une même photo : essayer `cv2.adaptiveThreshold()`. Si des segments sont coupés : essayer `cv2.dilate()` ou `cv2.morphologyEx()` pour les recoller.

C'est réussi quand : sur toutes les photos de réglage, on voit des chiffres blancs nets et un fond noir propre.

Étape 3 — Découper chaque chiffre (B)

Deux méthodes, essayer la plus simple d'abord :

1. **Contours** : `cv2.findContours()` puis `cv2.boundingRect()` pour avoir un rectangle par chiffre. Trier les rectangles de gauche à droite (par leur `x`). Ignorer les rectangles trop petits (le point décimal, les poussières).
2. **Colonnes** : si tous les chiffres ont la même largeur, couper l'image en N parts égales.

Piège : sur beaucoup de compteurs, les chiffres sont **penchés** (en italique) et se touchent. Piste : redresser l'image avec `cv2.getAffineTransform()` + `cv2.warpAffine()` (un « cisaillement ») avant de découper.

Le **point décimal** : soit on le détecte (petit contour en bas), soit on sait d'avance combien il y a de décimales sur ce compteur (le plus simple).

Étape 4 — Reconnaître un chiffre (C)

Commencer par la méthode la plus simple, passer à la suivante seulement si ça ne suffit pas.

Méthode 1 : ssocr (outil tout prêt)

`ssocr` = *Seven Segment Optical Character Recognition*. Un programme fait exprès pour les afficheurs 7 segments. On l'appelle depuis Python avec `subprocess.run()`. Tesseract, lui, est **mauvais** sur ce type de chiffres.

Méthode 2 : les 7 segments (à coder soi-même, recommandé)

Chaque chiffre est fait de 7 segments, nommés a à g :

```
 aaa
f   b
 ggg
e   c
 ddd
```

Pour chaque chiffre découpé :

1. Définir 7 petites zones, une par segment.
2. Pour chaque zone, compter les pixels blancs (`cv2.countNonZero()`). Si plus de ~50 % sont blancs, le segment est **allumé**.
3. Comparer la combinaison obtenue à une table (un dictionnaire Python) :

Chiffre	Segments allumés
0	a b c d e f
1	b c
2	a b d e g
3	a b c d g
4	b c f g
5	a c d f g
6	a c d e f g
7	a b c
8	a b c d e f g
9	a b c d f g

Attention : le « 1 » est très étroit. Un rectangle beaucoup plus fin que les autres = c'est sûrement un 1.

Méthode 3 : comparaison avec des modèles

Garder une image propre de chaque chiffre 0–9, puis comparer avec `cv2.matchTemplate()`. Plus sensible à la taille et à l'angle.

Étape 5 — Assembler le nombre (C)

Coller les chiffres dans l'ordre, ajouter la virgule à la bonne place, convertir en `float`. Si un seul chiffre n'est pas reconnu → renvoyer « rien » (`None`) plutôt qu'un faux nombre.

Pour déboguer

- Sauver les images intermédiaires (`cv2.imwrite()`) : gris, seuillé, chaque chiffre découpé. 90 % des bugs se voient à l'œil.
- Ces images intermédiaires sont **parfaites pour le poster** : elles montrent la méthode.

Le site Flask (rôle D)

Le site est déjà commencé dans `app.py` (classe `Application`) et `templates/index.html` (graphique Chart.js en mode test).

Comment ça marche

1. Le navigateur demande `/` → Flask renvoie la page HTML (`render_template()`).
2. Le JavaScript de la page appelle `/api/data` avec `fetch()`.
3. Flask lit la base, calcule, et renvoie du **JSON**.
4. Le JavaScript remplit les chiffres et dessine le graphique avec Chart.js.

Avantage : le Pico (plan B) pourra parler au même site sans rien changer.

Ce que `/api/data` doit renvoyer

La page de votre collègue attend déjà ce format (voir `DONNEES_TEST` dans `index.html`) :

Clé	Contenu
`points`	liste d'intervalles, chacun avec `t` (date en texte ISO) et `conso` (kWh)
`stats.moyenne`	moyenne des consommations
`stats.max` et `stats.max_t`	le pic et sa date
`stats.total`	consommation totale

À **ajouter** : `stats.min` (le minimum est demandé), et si possible le taux de réussite et la liste des lectures rejetées.

Indices Flask :

- Une méthode de route qui renvoie un `dict` → Flask le transforme en JSON tout seul (sinon `jsonify()`).
- Lire un paramètre d'URL (`/api/data?source=vrai`) : `request.args.get()`.
- Les routes sont déjà déclarées avec `add_url_rule()` dans `_enregistrer_routes()`.

Les calculs (classe `AnalyseConsommation`)

- Trier les relevés par date (`sorted()` avec `key=`).
- Pour chaque relevé à partir du 2e : `conso = index − index précédent`.
- **Garde-fou** : si `conso < 0`, la lecture est fausse → on la met dans les rejets et on ne l'utilise pas.
- Moyenne : `sum()` / `len()`. Minimum et pic : `min()`, `max()` (avec `key=` pour garder la date du pic).
- Arrondir avec `round(x, 2)` pour l'affichage.

Astuce : avancer sans attendre la lecture automatique

Prévoir un paramètre `source` :

- `source=vrai` → les calculs utilisent `vrai_index` (noté à la main).

- `source=lu` → ils utilisent `index_lu` (le programme).

Comme ça, le site est **fini et testable dès la semaine 1**, même si B et C n'ont pas fini. Et pour la démo, on peut montrer les deux courbes et comparer.

La page (Chart.js)

- Passer `MODE_TEST` à `false` quand `/api/data` marche.
- Ajouter une carte pour le **minimum**.
- Mettre le **pic** en évidence : un 3e jeu de données (*dataset*) avec un seul point, ou l'option `pointRadius` / `pointBackgroundColor` qui change pour ce point.
- Déplacer le script dans `static/js/script.js` et le lier avec `url_for('static', filename='js/script.js')`.
- Doc Chart.js à lire : *Line Chart*, *Datasets*, *Point Styling*, *Axes › Title*.

Lancer

- `python app.py` puis ouvrir `http://127.0.0.1:5000`.
- `debug=True` recharge le site à chaque modification du code.

Ressources à lire

Python et POO

- Classes : docs.python.org — Classes
- `@dataclass` : docs.python.org — dataclasses
- Lire un CSV : docs.python.org — csv (`DictReader`)
- Dates : docs.python.org — datetime (`fromisoformat`, `isoformat`)

Traitement d'image (OpenCV)

- Tutoriels OpenCV-Python, surtout :
- *Image Thresholding* (seuillage, Otsu, adaptatif)
- *Smoothing Images* (flou gaussien)
- *Contours : Getting Started* et *Contour Features* (`findContours`, `boundingRect`)
- *Morphological Transformations* (`dilate`, `erode`)
- *Geometric Transformations* (`warpAffine`, pour redresser)
- *Template Matching* (méthode 3)
- ssocr : chercher « ssocr seven segment » (page officielle de l'outil, avec ses options).
- Mots-clés utiles : « seven segment display recognition opencv python ».

Base de données

- Supabase — Tables
- Supabase — Se connecter à Postgres
- psycopg 3 — Basic usage
- supabase-py (si on choisit la bibliothèque Supabase)

Site

- Flask — Quickstart : routes, `render_template`, `request`, fichiers statiques, JSON
- Chart.js — Line Chart
- MDN — Utiliser fetch

Pico (plan B)

- MicroPython — Raspberry Pi Pico W (réseau)
- Documentation Arducam de **votre modèle** de caméra (exemples pour Pico).

Poster

- Consignes et exemples sur Moodle (lien « Conseils pour la réalisation d'un poster »).
- Outils : Canva ou PowerPoint (format A1 portrait : 841 × 594 mm).